WorkGrid — Project Constitution

Newtonite Software Engineering Challenge, 'Operations Under Pressure'. One-day full-stack assessment. This constitution holds the rules, the design and the build order that everything else must follow. Any change goes in the amendments log at the end.

The rules that never bend

1. One identity per person. Nobody needs a separate account for each job or team.
2. Roles are scoped to an organization, a department or a team. A person holds at most one role per place and can hold different roles in different places.
3. A role, a role request and a role assignment are three different things. A request is intent. Only an active assignment grants access.
4. Pending and rejected requests grant nothing. A pending promotion never removes access the person already has.
5. Every request is decided on its own. Approving one never approves another.
6. Nobody decides their own request or approves an item they own.
7. The server decides every access question on every request, denying by default. The interface hides things for convenience only.
8. Access is read from current assignments, never cached in a token. Suspension or revocation applies from the next request after it commits, even in an old browser tab.
9. What lies outside a person's scope is invisible to them: 404, never 403.
10. Every approval, rejection, role change, revocation and work-item change is recorded in append-only history with its reason.
11. Changes that must not be lost or doubled happen in one database transaction. Slow or fallible work, such as notifications, happens afterwards through the outbox.
12. Retries are safe. Mutating requests accept an Idempotency-Key and background handlers tolerate running twice.
13. A smaller system whose behaviour is proven by tests beats a larger one that is not.

1. What WorkGrid is

WorkGrid replaces chat threads, spreadsheets and email for coordinating operational work in a growing company. Teams create work items (an incident, a payment investigation, a customer issue, a compliance request). Each item has an owner, a priority,

a status and a full timeline. The landing screen is a 'needs attention' queue, not a pile of everything.

The design target is the brief, not Newtonite's real size: thousands of users, hundreds to thousands online at once, many teams, tens of thousands of active items and a very large history. The same design works for one small team.

Assumptions

- Work items belong to exactly one team.
- One identity per person. A person holds at most one role per scope node (organization, department or team) and can hold different roles in different places.
- A person's role in a team is the highest active assignment covering it: on the team, on its department or on its organization.
- Role requests are decided in one step by an eligible approver chosen from a policy table. Owners are the fallback authority.
- Demo data (organization, departments, teams, people, items) is invented and says so in the README. One organization is seeded and the model supports more.
- Single region, single database, no external email or chat delivery in version one.

Deliberately not built

Email or Slack delivery, file attachments, SSO and MFA, an SLA rules engine, approval chains with several approvers per request, expiry of requests and assignments, documents and reports modules, Kafka or Redis, typo-tolerant search. Each gets one line in the decisions document with the reason.

2. Scope tree, roles and assignments

The company is a tree: an organization contains departments and a department contains teams. A role is held at one node of the tree and applies to everything beneath it.

Role	Held at	Rank	What it can do
viewer	any node	0	Read items and history
member (employee, team member)	team	1	Create, comment, claim unowned items, edit and change the status of items they created or own

Role	Held at	Rank	What it can do
team_leader	team	2	Everything a member can, plus edit any item, assign work, change any item's status, approve or reject items in the team
department_manager	department	3	Team leader powers in every team of the department
organization_head	organization	4	Team leader powers across the organization
owner	organization	5	Governance authority: all of the above, decides organization head requests, and is the fallback approver

Three things stay apart. A role is the position. A role request is an application for it, waiting for a decision. A role assignment is the approved, active membership that actually grants access. Only an active assignment grants access.

Effective role. A person's role in a team is the highest active assignment covering it: on the team, on its department or on its organization. It is computed from current assignments on every request and is never stored in the login token.

One role per place. A person has at most one active assignment per node of the tree and can hold different roles in different places: member in Engineering, team leader in Operations, viewer in Design. In the demo data Rahul is team leader in Engineering and a member in Operations.

- Authorization is enforced by the server on every request. Hidden buttons are convenience only.
- Access starts from the item's team. Being logged in, or belonging elsewhere, grants nothing.
- Nobody can approve or reject an item they own, whatever their role.

3. Role requests and approvals

A person asks for a role at a place. An eligible approver decides. Only approval creates an assignment, and only inside the same transaction as the decision.

Requested role at	Decided by (or higher, covering that place)
member at a team	team_leader of that team
team_leader at a team	department_manager of the team's department
department_manager at a department	organization_head of its organization
organization_head at an organization	owner of that organization
owner at an organization	another existing owner
viewer at any place	team_leader or higher covering that place (a policy row, changeable)

Fallback: if nobody eligible exists, owners can decide the request. Nothing is ever approved silently. The table lives in the approval_policies table, so changing routing is data, not code.

Lifecycle: pending, then approved, rejected or cancelled. Only the requester can cancel, and only while pending.

The approval transaction:

1. Lock the request row, then confirm it is still pending.
2. Confirm the decider holds an active, covering role of enough rank and is not the requester. Authority is checked at decision time, not at request time.
3. Write the approval record with decider, outcome, comment and timestamp.
4. On approval, replace any existing assignment on that same place (a promotion) and create the new active assignment.
5. Write the audit entry and the outbox job. Commit everything together.

Safety rules:

- Two simultaneous deciders cannot both win: the second waits on the row lock, then sees a decided request and gets 409.
- Retries are safe through the Idempotency-Key and the status check.
- A person has at most one pending request per place and requested role.
- Routing only moves upward in rank, and owner requests need an existing owner, so no approval chain can loop. A test checks the policy table.
- Existing access stays active while any request is pending.

- Suspension, restoration and revocation need an authority above the holder, require a reason, take effect from the next request and are audited. A person's unfinished work stays with them until a team leader reassigns it. Nothing is silently transferred or deleted.

4. Tech stack and why

Layer	Choice	Why
Database	PostgreSQL 16	Transactions, row locking, optimistic versioning, full-text search, partial indexes and LISTEN/NOTIFY in one system. Most of the brief maps to features it already has.
API	Python 3.12, FastAPI	Clear request validation (Pydantic), automatic API docs, fast to build and easy to explain. Sync endpoints on a thread pool with a bounded connection pool.
DB access	psycopg 3 with psycopg_pool, hand-written SQL	The concurrency guarantees live in specific SQL statements. Writing them directly keeps them visible and defensible in review.
Auth	JWT (PyJWT), PBKDF2-SHA256 password hashing	Stateless API instances. Short expiry. Hashing from the standard library, no extra dependency.
Async work	Transactional outbox table plus a worker process	No new infrastructure. Jobs commit atomically with the change that caused them.
Live updates	Server-sent events fed by Postgres NOTIFY	One-way push is all we need. NOTIFY fires only on commit, so clients never hear about rolled-back changes.
Frontend	React, TypeScript, Vite, TanStack Query	Server state, caching, infinite lists and retries handled by one library instead of hand-rolled global state.

Layer	Choice	Why
Tests	pytest, httpx TestClient, real PostgreSQL	The risky behaviour is in the database interaction, so tests run against a real database, not mocks.
Packaging	Docker Compose (Postgres), venv for the API, npm for the web app	One command for the database, simple local run.

Why not the alternatives: MongoDB makes multi-document consistency and keyset queries harder. A message broker adds a service the scale does not justify. GraphQL and websockets add moving parts without helping the graded behaviours.

5. Architecture

React app talks JSON over HTTP to FastAPI, which talks SQL to PostgreSQL. A separate worker process reads the outbox table. An SSE endpoint streams change events that Postgres emits on commit.

- API instances are stateless, so several can run behind a load balancer.
- Every mutating request runs in exactly one database transaction: the change, the history entry, the outbox job, the idempotency record and the live-update event commit together or not at all.
- The worker is the same codebase run as a separate process, and can also be started inside the API for local demos.

6. Data model

Table	Purpose
users	Identity and password hash. One per person
organizations, departments, teams	The scope tree: an organization holds departments, a department holds teams
role_assignments	Granted access: user, role, scope type and id, status (active, suspended, revoked), who granted it, from which request

Table	Purpose
role_requests	Intent: user, requested role and scope, reason, status (pending, approved, rejected, cancelled)
approval_policies	Which role may decide which requested role, as data
approval_records	Every decision: decider, outcome, comment, timestamp
audit_log	Append-only record of access changes: grants, decisions, suspensions, revocations
work_items	The core record, described below
activity	Append-only timeline of each item. Comments live here too, so 'why is it like this' is one query
idempotency_keys	(user, key) primary key, request hash, stored response
outbox	Background jobs with status, attempts, next-attempt time, last error
notifications	In-app notifications with a unique dedupe key
effective_roles (view)	Highest active role per user per team, derived from assignments and the scope tree

Work item fields: id, team, kind (task, incident, customer_issue, payment, engineering, compliance), title, description, status, priority (low, normal, high, urgent), requires_approval, assignee, creator, due date, version, timestamps and a generated full-text search column.

Constraints that protect the rules even if application code has a bug:

- Check constraint: an item in progress or awaiting approval must have an owner.
- Check constraint on role and scope: member and team_leader at a team, department_manager at a department, organization_head and owner at an organization, viewer anywhere.
- Partial unique index: one active assignment per user per scope node, so one role per place.
- Partial unique index: one pending request per user, scope and requested role.

- Triggers on activity, audit_log and approval_records refuse update and delete. History cannot quietly change.
- Unique dedupe key on notifications: a job delivered twice cannot create two notifications.
- Primary key on idempotency keys: a request key executes at most once per user.

Indexes: team plus updated time plus id (main list), team plus status, partial index on assignee, GIN index on the search column, item plus id on activity, partial index on pending outbox rows.

Urgency is derived, not stored: an item is overdue when its due date has passed and it is not resolved or closed.

7. Work items and workflow

Statuses: open, in_progress, awaiting_approval, resolved, closed.

- open to closed: manager, reason required.
- Claiming or being assigned an open item moves it to in_progress.
- in_progress to resolved: only for items that do not require approval.
- in_progress to awaiting_approval: only for items that require approval.
- in_progress to open (release): owner or manager, reason required, clears the owner.
- awaiting_approval to resolved (approve) or to in_progress (reject, reason required): managers only, never the item's own owner.
- resolved to closed, or reopen to in_progress (reason required): owner or manager.
- closed is final.

One function decides what a given user may do to a given item in its current state. The API enforces it and the UI reads the same answer, so they cannot drift apart. Illegal moves return 422 with the list of allowed moves. Moves the user may not make return 403.

8. The behaviours that need more than CRUD

The brief requires at least three. We implement and test six.

#	Problem	Mechanism	Proof
1	Two people take the same work	One UPDATE conditional on the item being open and unowned. The database serializes competing updates, so exactly one succeeds. Losers get 409 naming the owner.	12 simultaneous claims, one winner, one history entry
2	Editing stale information	Every item has a version. Edits and transitions send the version they saw and the update is conditional on it. A mismatch returns 409 with the current item.	Second edit from the same version is rejected, newer data kept
3	Retries and double clicks	Idempotency-Key. The key row is inserted in the same transaction as the action, so a parallel duplicate waits on the key, then replays the stored response. A failed request leaves no key, so a retry can succeed.	8 parallel identical requests create one comment
4	Workflow and approval rules	Rules as data, enforced server-side, plus owner-cannot-approve	Self-approval refused, even by a manager who owns the item
5	Resource-level access	Every read starts from the caller's effective role in the item's team	Non-members get 404 on read, claim, comment, history and create
6	Reliable background work	Outbox row in the same transaction as the change. Worker takes rows with FOR UPDATE SKIP LOCKED, retries with backoff, dead after five attempts. Handlers are idempotent by dedupe key.	Failing handler retries, duplicate delivery gives one notification

#	Problem	Mechanism	Proof
7	Two approvers decide one request	Request row locked FOR UPDATE. Status and authority are re-checked inside the lock, so the second decider gets 409.	10 simultaneous approvals create one assignment and one approval record
8	Access that changes mid-session	Permissions are read from active assignments on every request, never from the token. Pending requests grant nothing. A promotion keeps the old role until approved.	Revoked user's old token is refused on the next call

Items 1 to 5 are built and tested on the current role model. Items 6 to 8 are built in phases 2 and 3 of section 16.

9. API

All errors use one shape: code, message and, for conflicts, the current item. Status codes: 401 not signed in, 403 not permitted, 404 not found or not visible, 409 conflict (stale version or already claimed), 422 invalid (illegal transition, missing reason, validation).

- Session: POST /auth/login, GET /me, GET /me/positions (active assignments with scope names)
- Dashboard: GET /me/dashboard (name, positions with counts, my requests, unread count), GET /me/todo (unified list across positions, server-ordered)
- Items: GET /items with filters (scope, view: all, mine, approvals, unassigned, overdue; status, priority, kind, assignee, q, cursor, limit), GET /items/summary, POST /items, GET and PATCH /items/{id}, POST /items/{id}/claim, /assign, /transition, /comments, GET /items/{id}/activity
- Role requests: POST /role-requests, GET /me/role-requests, POST /role-requests/{id}/cancel
- Approvals: GET /approvals/inbox, POST /role-requests/{id}/approve and /reject (reject needs a reason)
- Assignments: GET /scopes/{type}/{id}/assignments, POST /assignments/{id}/suspend, /restore, /revoke (reason required)

- Scope and workspace: GET /scopes, GET /workspaces/{assignment_id}, GET /scopes/{type}/{id}/members, GET /scopes/{type}/{id}/audit
- Notifications and live updates: GET /notifications, POST /notifications/{id}/read, GET /events (SSE)

Mutating routes accept an optional Idempotency-Key header. Reasons are required on reassign, release, reject and reopen and are stored in the history.

10. Search, pagination and scale

- Lists use keyset pagination on (updated time, id) with an opaque cursor. There are no offsets anywhere, so page 500 costs the same as page 1.
- Every list query is scoped by team membership and uses an index.
- Search is PostgreSQL full-text on title and description, combined with filters. Results stay in recency order so cursors remain stable. Relevance ranking is a noted trade-off.
- Activity is paginated per item with its own index.
- The browser only ever holds the pages the user has scrolled through.
- Planned: a load script that generates about 5,000 users, 50 teams, 50,000 items and a large history, with EXPLAIN output for the main queries recorded in the decisions document.
- Known cost to mention in review: summary counts scan a team's items. At much larger scale they would move to maintained counters.

11. Dashboard, workspaces and frontend

One dashboard per person, however many positions they hold. Everything on it comes from the server's answer about their active assignments.

- Header: greeting by name from the session, date, notifications bell.
- Position cards: one per ACTIVE assignment, showing role, place, open and overdue counts. A card opens that workspace. Pending, rejected, cancelled, suspended and revoked roles never appear as cards.
- My requests: every request with its status. A request is never counted as access.
- Unified to-do list: items assigned to me across all positions, each labelled with team and role. Priority (low, normal, high, urgent) is business importance. Urgency is derived from the due date. Order: overdue (by priority, then days overdue), due within 24 hours, then priority, then deadline, then id as a stable tie-breaker. The server orders it.

- Workspace for one assignment: bound to that place. Editing a URL cannot widen it because the server re-checks. Modules: Overview (counts, what needs attention), Work (list, search, filters, detail, timeline), People (who holds which role here), Requests (inbox, shown only if the role may decide), Activity. Documents, reports and performance modules are out of scope.
- Approvals inbox: pending requests this person is eligible to decide, with requester, role, place and reason. Approve, or reject with a reason.
- Behaviour: TanStack Query with infinite lists over keyset cursors, filters held in the URL, optimistic claim that rolls back on 409 and shows who won, a conflict screen that shows the latest values beside the user's edit, one idempotency key per action reused on retry, success shown only after the server confirms.
- Live updates: SSE says which items changed, the client refetches only what is on screen and shows a 'new updates' banner instead of shuffling the list. Polling on focus is the fallback.
- Quality: loading, empty, error, forbidden and expired-session states, visible keyboard focus, labelled controls, priority and status shown in words as well as colour.
- Look: plain and fast. A coloured rail on each row for priority, one strong signal colour for urgency, quiet neutrals, no decoration, no copied branding.

12. Testing strategy

Tests target what would be dangerous if wrong, against a real database.

- Concurrency: claim race, stale edits, replayed and parallel duplicate requests, failed request not cached, key reused for a different request. (Done)
- Authorization: non-member access on every route, viewer read-only, who may assign and edit, different roles in different teams, assignee eligibility, unauthenticated calls. (Done, to be re-run on the new model)
- Workflow: legal and illegal moves, approval cannot be skipped, owner cannot approve own item, reasons required, release clears owner, stale transition conflicts, database refuses an in-progress item without an owner. (Done)
- History and lists: history append-only at database level, one entry per change with its reason, activity and list pagination without gaps, filters, search, overdue view. (Done)
- Access model: effective role is the highest covering active assignment, one active assignment per place, pending, rejected and cancelled requests grant nothing, a promotion keeps the old role until approved, suspension and revocation refuse an old token on the next call, role and scope pairing enforced. (To do)

- Approvals: self-approval refused, an approver without authority for that exact place refused, two requests decided independently, 10 simultaneous approvals give one assignment, idempotent retry, authority re-checked at decision time, policy table has no cycle. (To do)
- Worker: failure and backoff, dead after five attempts, duplicate delivery gives one notification, two workers never take one job, unfinished job recovered. (To do)
- Dashboard: to-do ordering, positions show only active assignments, workspace scope cannot be widened by editing ids. (To do)
- One end-to-end smoke test of the main journey through the running API. (To do)

13. Repository layout

workgrid/

- README.md, ENGINEERING_DECISIONS.md, LIMITATIONS.md, docker-compose.yml
- backend/app: main, items, authz, workflow, idempotency, security, db, config, errors, seed, schema.sql. To add: access (scope tree and effective roles), requests (approval engine), dashboard, worker, hub, audit, load seed
- backend/tests: concurrency, authz, workflow, history and lists. To add: access, approvals, worker, dashboard, smoke
- frontend/src: api client, views (dashboard, workspace, approvals), components

14. How to run (target instructions for the README)

1. Start PostgreSQL (Docker Compose file to be added) and create the database.
2. In backend: create a virtual environment, install requirements, set DATABASE_URL.
3. Seed demo data with python -m app.seed. Every demo user has the password 'workgrid'.
4. Start the API with uvicorn app.main:app.
5. In frontend: npm install, npm run dev, open the printed address.
6. Run tests from backend with pytest.

Demo users are re-seeded in phase 1 to match the scope tree: an owner, a department manager, team leaders, members and a viewer, with different roles in different teams. Every demo user has the password 'workgrid'.

15. Documents to deliver

ENGINEERING_DECISIONS.md, five decisions, each with problem, choice, alternatives, trade-off and what changes at ten times the scale:

1. PostgreSQL as the single store for data, queue and events.
2. Optimistic versioning plus atomic claim for concurrency.
3. Idempotency keys stored in the same transaction as the action.
4. Transactional outbox for notifications.
5. Scoped roles granted by approved assignments, with server-side resource checks and database constraints as a safety net.

Each decision names the Newtonite principle it follows: useful before impressive, systems over isolated features, ambition grounded in real constraints.

LIMITATIONS.md: no email or chat delivery, no role-request workflow, single region, SSE fan-out limits per process, search is not typo tolerant or ranked, summary counts scan rather than use counters, basic auth without MFA, and the questions to ask Newtonite on day one (how many teams, who approves what, where requests come from today).

Demo script, about five minutes: create and assign an item, change priority and show history with reasons, act as a viewer and see the refusal, submit a stale edit and see the conflict, double-submit and see one result, show a failing background job retrying.

16. Implementation phases and status

Phase	Work	Done when	Status	Effort
0	Foundation: schema, auth, items, workflow, idempotency, history, 26 tests	Tests green	Done	n/a
1	Access model: scope tree, role_assignments, effective_roles view, role ranks and scope pairing, audit_log, migrate seed and tests, positions endpoint	All existing tests pass on the new model and the access tests pass	Next	1.5h

Phase	Work	Done when	Status	Effort
2	Role requests and approvals: policy table, request lifecycle, approval transaction, cancel, suspend, restore, revoke, inbox	Self-approval, scope, independence, concurrent approval and revocation tests pass	Planned	2.5h
3	Outbox worker and notifications for item and access events	Retry, dead-letter, duplicate delivery and two-worker tests pass	Planned	1.5h
4	Dashboard API: /me/dashboard, /me/todo ordering, workspace and scope filters, summaries	Ordering and scope tests pass	Planned	1h
5	Live updates over SSE	A change in one session appears in another	Planned	0.75h
6	React frontend: login, dashboard, workspace, approvals inbox, notifications	The demo script runs end to end by hand	Planned	4h
7	Load seed and query timing	EXPLAIN recorded for the main queries	Planned	0.75h
8	Docker Compose, README, decisions, limitations, demo script, end-to-end smoke test	A clean machine runs it from the README	Planned	1.5h

About 13.5 hours remain. Each phase ends with every test green before the next begins. Cut order if time runs short: the load-seed polish (7), then SSE falls back to polling (5), then workspace modules beyond Overview, Work and People. Never cut the tests, the approval engine's safety rules or the decisions document.

17. Questions to be ready for in the review

- What happens if the worker dies mid-job? (The job stays pending or is recovered by timeout, handlers are idempotent, so reprocessing is safe.)
- Why not Redis or Kafka? (Scale does not justify another service. The outbox gives the same atomicity guarantee with what we already run.)

- What breaks at ten times the load? (Summary counts, SSE fan-out per process, history table size. Mitigations: counters, a shared pub/sub layer, partitioning activity by time.)
- Why 404 instead of 403 for other teams? (Avoids leaking that an item exists.)
- What does idempotency do if the first request failed? (Nothing is stored, so the retry runs normally.)
- Which part is least safe? (Basic auth and token handling, no rate limiting, no MFA. Named honestly in the limitations.)
- What next with another week? (Role-request workflow, notification channels, ranked search, counters, audit log for membership changes, load testing in CI.)

Also be ready for: why a role is computed from assignments instead of stored on the user (so revocation is immediate and nothing can go stale), why a pending request grants nothing (a request is intent, not authority), and what happens when an approver loses authority while a request is pending (the check is made at decision time, so they cannot decide it, and the request stays pending for others).

18. Risks

- Scope growth: the access model and approval engine are the biggest additions. Mitigation: one-step approvals, a fixed policy table, no expiry, and phases gated by green tests.
- Time: the frontend is the largest single piece. Mitigation: dashboard, one workspace layout and an inbox, reused everywhere.
- Concurrency bugs that only show under load: parallel tests cover the main paths and the load script adds more.
- Over-explaining: the review rewards clear reasoning about a smaller system, so the decisions document stays short and honest.

19. Amendments log

Every change to this constitution is recorded here with its reason.

Date	Change	Reason
2026-10-17	Added the scope tree, scoped role assignments, role requests and the approval hierarchy (sections 2, 3)	A person can hold different roles in different places, granted through an approval hierarchy

Date	Change	Reason
2026-10-17	Rebuilt phases around the access model (section 16)	The access model changes what everything else is built on
2026-10-17	Renamed Opsboard to WorkGrid	Final product name
2026-10-16	Chose FastAPI, PostgreSQL and React	Candidate's stack choice